

SRI SRI UNIVERSITY

BANK MANAGEMENT SYSTEM

Complete Source Code in Java and Python

Second Year BTech Project

Karan Sanjay Yadav
Ayushman Barik
Nishit Modi
Ashwin Kumar Nayak
Kaushik Shyamal

Contents

Part 1: Java Source Code 3

File: Account.java 3

File: SavingsAccount.java 4

File: CurrentAccount.java 4

File: Bank.java 5

File: Transaction.java 6

File: BankException.java 6

File: InvalidAmountException.java 6

File: InsufficientFundsException.java 6

File: AccountNotFoundException.java 6

File: CsvProcessor.java 7

File: BankApp.java 9

File: TestRunner.java 15

File: Simulation.java 16

Part 2: Python Source Code 18

File: bank_management_system.py 18

About this listing

This document contains the complete source code of the Bank Management System. The Java version is a desktop application with a Swing interface and CSV import; the Python version runs from the command line and can process the same CSV file. Both versions use the same four classes - **Account** (abstract), **SavingsAccount**, **CurrentAccount** and **Bank** - and apply the same rules, so the sample file `sample_transactions.csv` gives identical results in both: 16 rows, 12 successful, 4 rejected and a total balance of Rs. 83,000.

Savings accounts must keep a minimum balance of Rs. 500. Current accounts may go into an overdraft of up to Rs. 10,000.

How to run: see the README.md file in the project folder.

Part 1: Java Source Code

File: Account.java

```
 1 | import java.util.ArrayList;
 2 | import java.util.Collections;
 3 | import java.util.List;
 4 |
 5 | /**
 6 |  * Abstract base class: defines the common data and interface of every account.
 7 |  * Demonstrates ENCAPSULATION (private fields) and ABSTRACTION (abstract methods).
 8 |  */
 9 | public abstract class Account {
10 |
11 |     // ---- Encapsulation: all data is private ----
12 |     private final String accountNumber;
13 |     private final String holderName;
14 |     private double balance;
15 |     private final List<Transaction> transactions = new ArrayList<Transaction>();
16 |
17 |     protected Account(String accountNumber, String holderName, double openingBalance)
18 |             throws InvalidAmountException {
19 |         if (openingBalance < 0) {
20 |             throw new InvalidAmountException("Opening balance cannot be negative.");
21 |         }
22 |         this.accountNumber = accountNumber;
23 |         this.holderName = holderName;
24 |         this.balance = openingBalance;
25 |         log("Opening", openingBalance, "Success");
26 |     }
27 |
28 |     // ---- Read-only access through getters (no setter for balance) ----
29 |     public String getAccountNumber() { return accountNumber; }
30 |     public String getHolderName()    { return holderName; }
31 |     public double getBalance()       { return balance; }
32 |
33 |     public List<Transaction> getTransactions() {
34 |         return Collections.unmodifiableList(transactions);
35 |     }
36 |
37 |     // ---- Behaviour shared by all accounts ----
38 |     public double deposit(double amount) throws InvalidAmountException {
39 |         validate(amount);
40 |         balance += amount;
41 |         log("Deposit", amount, "Success");
42 |         return balance;
43 |     }
44 |
45 |     // ---- Protected helpers for subclasses ----
46 |     protected double debit(double amount) {
47 |         balance -= amount;
48 |         log("Withdrawal", amount, "Success");
49 |         return balance;
50 |     }
51 |
52 |     protected void reject(double amount) {
53 |         log("Withdrawal", amount, "Rejected");
54 |     }
55 |
56 |     protected void validate(double amount) throws InvalidAmountException {
57 |         if (amount <= 0) {
58 |             throw new InvalidAmountException("Amount must be greater than zero.");
59 |         }
60 |     }
61 |
62 |     private void log(String type, double amount, String status) {
63 |         transactions.add(new Transaction(type, amount, status, balance));
64 |     }
65 |
66 |     // ---- Abstraction: every subclass MUST implement these ----
67 |     public abstract double withdraw(double amount) throws BankException;
```

```
68 |
69 |     public abstract String getAccountType();
70 |
71 |     @Override
72 |     public String toString() {
73 |         return String.format("%-8s | %s | %-22s | Rs. %,12.2f",
74 |                 getAccountType(), accountNumber, holderName, balance);
75 |     }
76 | }
```

File: SavingsAccount.java

```
 1 | /** Savings account: earns interest and must keep a minimum balance. */
 2 | public class SavingsAccount extends Account {
 3 |
 4 |     private final double interestRate;     // yearly rate, e.g. 0.04 = 4 %
 5 |     private final double minimumBalance;
 6 |
 7 |     public SavingsAccount(String accountNumber, String holderName, double openingBalance)
 8 |             throws InvalidAmountException {
 9 |         super(accountNumber, holderName, openingBalance);
10 |         this.interestRate = 0.04;
11 |         this.minimumBalance = 500.0;
12 |     }
13 |
14 |     @Override   // Polymorphism: the savings-account rule
15 |     public double withdraw(double amount) throws BankException {
16 |         validate(amount);
17 |         if (getBalance() - amount < minimumBalance) {
18 |             reject(amount);
19 |             throw new InsufficientFundsException(String.format(
20 |                     "Minimum balance of Rs. %,.0f must be maintained.", minimumBalance));
21 |         }
22 |         return debit(amount);
23 |     }
24 |
25 |     /** Credits one month of interest and returns the amount credited. */
26 |     public double addInterest() throws InvalidAmountException {
27 |         double interest = getBalance() * interestRate / 12;
28 |         deposit(interest);
29 |         return interest;
30 |     }
31 |
32 |     @Override
33 |     public String getAccountType() {
34 |         return "Savings";
35 |     }
36 | }
```

File: CurrentAccount.java

```
 1 | /** Current account: allows withdrawals up to an overdraft limit. */
 2 | public class CurrentAccount extends Account {
 3 |
 4 |     private final double overdraftLimit;
 5 |
 6 |     public CurrentAccount(String accountNumber, String holderName, double openingBalance)
 7 |             throws InvalidAmountException {
 8 |         super(accountNumber, holderName, openingBalance);
 9 |         this.overdraftLimit = 10000.0;
10 |     }
11 |
12 |     @Override   // Polymorphism: the current-account rule
13 |     public double withdraw(double amount) throws BankException {
14 |         validate(amount);
15 |         if (getBalance() - amount < -overdraftLimit) {
16 |             reject(amount);
17 |             throw new InsufficientFundsException(String.format(
18 |                     "Overdraft limit of Rs. %,.0f exceeded.", overdraftLimit));
19 |         }
20 |         return debit(amount);
21 |     }
22 |
```

```
23 |     @Override
24 |     public String getAccountType() {
25 |         return "Current";
26 |     }
27 | }
```

File: Bank.java

```
 1 | import java.util.ArrayList;
 2 | import java.util.LinkedHashMap;
 3 | import java.util.List;
 4 | import java.util.Map;
 5 |
 6 | /** The bank: owns every account and coordinates all operations. */
 7 | public class Bank {
 8 |
 9 |     private final String bankName;
10 |     private final Map<String, Account> accounts = new LinkedHashMap<String, Account>();
11 |     private int nextNumber = 1001;
12 |
13 |     public Bank(String bankName) {
14 |         this.bankName = bankName;
15 |     }
16 |
17 |     public String getBankName() {
18 |         return bankName;
19 |     }
20 |
21 |     public Account createAccount(String type, String holderName, double openingBalance)
22 |             throws BankException {
23 |         String number = "ACC" + nextNumber;
24 |         Account account;
25 |         if ("savings".equalsIgnoreCase(type)) {
26 |             account = new SavingsAccount(number, holderName, openingBalance);
27 |         } else if ("current".equalsIgnoreCase(type)) {
28 |             account = new CurrentAccount(number, holderName, openingBalance);
29 |         } else {
30 |             throw new BankException("Account type must be SAVINGS or CURRENT.");
31 |         }
32 |         accounts.put(number, account);
33 |         nextNumber++;
34 |         return account;
35 |     }
36 |
37 |     public Account findAccount(String accountNumber) throws AccountNotFoundException {
38 |         Account account = accounts.get(accountNumber);
39 |         if (account == null) {
40 |             throw new AccountNotFoundException("Account " + accountNumber + " not found.");
41 |         }
42 |         return account;
43 |     }
44 |
45 |     public double deposit(String accountNumber, double amount) throws BankException {
46 |         return findAccount(accountNumber).deposit(amount);
47 |     }
48 |
49 |     public double withdraw(String accountNumber, double amount) throws BankException {
50 |         return findAccount(accountNumber).withdraw(amount);   // polymorphic call
51 |     }
52 |
53 |     public double checkBalance(String accountNumber) throws AccountNotFoundException {
54 |         return findAccount(accountNumber).getBalance();
55 |     }
56 |
57 |     public List<Account> getAllAccounts() {
58 |         return new ArrayList<Account>(accounts.values());
59 |     }
60 |
61 |     public double getTotalBalance() {
62 |         double total = 0;
63 |         for (Account account : accounts.values()) {
64 |             total += account.getBalance();
65 |         }
66 |         return total;
```

```
    }
}
```

File: Transaction.java

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

/** One entry in the history of an account. */
public class Transaction {
    private static final DateTimeFormatter FORMAT =
            DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    private final String time;
    private final String type;
    private final double amount;
    private final String status;
    private final double balanceAfter;

    public Transaction(String type, double amount, String status, double balanceAfter) {
        this.time = LocalDateTime.now().format(FORMAT);
        this.type = type;
        this.amount = amount;
        this.status = status;
        this.balanceAfter = balanceAfter;
    }

    public String getTime()        { return time; }
    public String getType()        { return type; }
    public double getAmount()      { return amount; }
    public String getStatus()      { return status; }
    public double getBalanceAfter() { return balanceAfter; }
}
```

File: BankException.java

```
/** Base class for every banking related error (checked exception). */
public class BankException extends Exception {
    public BankException(String message) {
        super(message);
    }
}
```

File: InvalidAmountException.java

```
/** Thrown when an amount is zero, negative or otherwise not acceptable. */
public class InvalidAmountException extends BankException {
    public InvalidAmountException(String message) {
        super(message);
    }
}
```

File: InsufficientFundsException.java

```
/** Thrown when a withdrawal breaks the rules of the account type. */
public class InsufficientFundsException extends BankException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}
```

File: AccountNotFoundException.java

```
/** Thrown when an account number does not exist in the bank. */
public class AccountNotFoundException extends BankException {
    public AccountNotFoundException(String message) {
        super(message);
    }
}
```

File: CsvProcessor.java

```
 1 | import java.io.BufferedReader;
 2 | import java.io.File;
 3 | import java.io.FileReader;
 4 | import java.io.IOException;
 5 | import java.util.ArrayList;
 6 | import java.util.List;
 7 |
 8 | /**
 9 |  * Reads a CSV file and feeds it to the bank.
10 |  * Expected header: Action,AccountType,HolderName,AccountNumber,Amount
11 |  * Actions: CREATE, DEPOSIT, WITHDRAW, BALANCE
12 |  */
13 | public class CsvProcessor {
14 |
15 |     /** Outcome of one CSV row. */
16 |     public static class Result {
17 |         public final int row;
18 |         public final String action;
19 |         public final String account;
20 |         public final String amount;
21 |         public final String status;
22 |         public final String message;
23 |
24 |         Result(int row, String action, String account, String amount, String status, String message) {
25 |             this.row = row;
26 |             this.action = action;
27 |             this.account = account;
28 |             this.amount = amount;
29 |             this.status = status;
30 |             this.message = message;
31 |         }
32 |     }
33 |
34 |     /** Splits one CSV line; understands "quoted, values". */
35 |     public static String[] parseLine(String line) {
36 |         List<String> fields = new ArrayList<String>();
37 |         StringBuilder current = new StringBuilder();
38 |         boolean quoted = false;
39 |         for (int i = 0; i < line.length(); i++) {
40 |             char c = line.charAt(i);
41 |             if (quoted) {
42 |                 if (c == '"' && i + 1 < line.length() && line.charAt(i + 1) == '"') {
43 |                     current.append('"');
44 |                     i++;
45 |                 } else if (c == '"') {
46 |                     quoted = false;
47 |                 } else {
48 |                     current.append(c);
49 |                 }
50 |             } else if (c == '"') {
51 |                 quoted = true;
52 |             } else if (c == ',') {
53 |                 fields.add(current.toString().trim());
54 |                 current.setLength(0);
55 |             } else {
56 |                 current.append(c);
57 |             }
58 |         }
59 |         fields.add(current.toString().trim());
60 |         return fields.toArray(new String[0]);
61 |     }
62 |
63 |     /** Reads every line of a CSV file (header included). */
64 |     public static List<String[]> readAll(File file) throws IOException {
65 |         List<String[]> rows = new ArrayList<String[]>();
66 |         BufferedReader reader = new BufferedReader(new FileReader(file));
67 |         try {
68 |             String line;
69 |             boolean first = true;
70 |             while ((line = reader.readLine()) != null) {
71 |                 if (first) {                    // remove the BOM written by Excel
72 |                     line = line.replace("\uFEFF", "");
73 |                     first = false;
```

```
                }
                if (!line.trim().isEmpty()) {
                    rows.add(parseLine(line));
                }
            }
        } finally {
            reader.close();
        }
        return rows;
    }

    /** Runs every row of the file against the given bank. */
    public static List<Result> process(File file, Bank bank) throws IOException {
        List<String[]> rows = readAll(file);
        List<Result> results = new ArrayList<Result>();
        for (int i = 1; i < rows.size(); i++) {             // row 0 is the header
            String[] f = rows.get(i);
            String action = field(f, 0).toUpperCase();
            String type = field(f, 1);
            String holder = field(f, 2);
            String number = field(f, 3);
            String amountText = field(f, 4);
            try {
                switch (action) {
                    case "CREATE": {
                        Account a = bank.createAccount(type, holder, Double.parseDouble(amountText));
                        results.add(new Result(i, action, a.getAccountNumber(), amountText, "Success",
                                "Created " + a.getAccountType() + " account for " + holder));
                        break;
                    }
                    case "DEPOSIT": {
                        double bal = bank.deposit(number, Double.parseDouble(amountText));
                        results.add(new Result(i, action, number, amountText, "Success",
                                String.format("New balance Rs. %,.2f", bal)));
                        break;
                    }
                    case "WITHDRAW": {
                        double bal = bank.withdraw(number, Double.parseDouble(amountText));
                        results.add(new Result(i, action, number, amountText, "Success",
                                String.format("New balance Rs. %,.2f", bal)));
                        break;
                    }
                    case "BALANCE": {
                        double bal = bank.checkBalance(number);
                        results.add(new Result(i, action, number, "-", "Success",
                                String.format("Balance is Rs. %,.2f", bal)));
                        break;
                    }
                    default:
                        results.add(new Result(i, action, number, amountText, "Rejected",
                                "Unknown action"));
                }
            } catch (NumberFormatException e) {
                results.add(new Result(i, action, number, amountText, "Rejected", "Amount is not a
                    number"));
            } catch (BankException e) {
                results.add(new Result(i, action, number, amountText, "Rejected", e.getMessage()));
            }
        }
        return results;
    }

    private static String field(String[] f, int index) {
        return index < f.length ? f[index] : "";
    }
}
```

File: BankApp.java

```
 1 | import javax.imageio.ImageIO;
 2 | import javax.swing.*;
 3 | import javax.swing.border.EmptyBorder;
 4 | import javax.swing.border.LineBorder;
 5 | import javax.swing.table.DefaultTableCellRenderer;
 6 | import javax.swing.table.DefaultTableModel;
 7 | import javax.swing.table.JTableHeader;
 8 | import java.awt.*;
 9 | import java.io.File;
10 | import java.util.ArrayList;
11 | import java.util.List;
12 |
13 | /** Desktop (Swing) interface of the Bank Management System. */
14 | public class BankApp extends JPanel {
15 |
16 |     static final Color NAVY = new Color(0x1F, 0x3A, 0x5F);
17 |     static final Color TEAL = new Color(0x2A, 0x9D, 0x8F);
18 |     static final Color LIGHT = new Color(0xEA, 0xF1, 0xF8);
19 |     static final Color GREEN = new Color(0x1B, 0x7F, 0x4B);
20 |     static final Color RED = new Color(0xC0, 0x39, 0x2B);
21 |
22 |     final Bank bank = new Bank("BTech National Bank");
23 |     final JTabbedPane tabs = new JTabbedPane();
24 |
25 |     // Dashboard
26 |     final JLabel lblAccounts = valueLabel(), lblSavings = valueLabel(),
27 |             lblCurrent = valueLabel(), lblTotal = valueLabel();
28 |     final DefaultTableModel accountModel =
29 |             model("Account No.", "Account Holder", "Type", "Balance (Rs.)");
30 |     final JLabel emptyHint = new JLabel(
31 |             "No accounts yet. Create one in the Operations tab or import a CSV file.",
   |                 SwingConstants.CENTER);
32 |
33 |     // Operations
34 |     final JComboBox<String> typeBox = new JComboBox<String>(new String[]{"Savings", "Current"});
35 |     final JTextField holderField = new JTextField(18);
36 |     final JTextField numberField = new JTextField(12);
37 |     final JTextField amountField = new JTextField(12);
38 |     final JButton btnCreate = button("Create Account", TEAL);
39 |     final JButton btnDeposit = button("Deposit", NAVY);
40 |     final JButton btnWithdraw = button("Withdraw", NAVY);
41 |     final JButton btnBalance = button("Check Balance", NAVY);
42 |     final JTextArea messageArea = new JTextArea();
43 |
44 |     // CSV import
45 |     final JTextField csvPathField = new JTextField();
46 |     final JButton btnChooseCsv = button("Choose CSV File...", TEAL);
47 |     final DefaultTableModel csvModel = model("#", "Action", "Account", "Amount (Rs.)", "Status",
   |         "Details");
48 |     final JLabel csvSummary = new JLabel("No file processed yet.");
49 |
50 |     // Specification viewer
51 |     final JTextField specPathField = new JTextField();
52 |     final JButton btnLoadSpec = button("Load Specification CSV...", TEAL);
53 |     final DefaultTableModel specModel = model("Load the specification file to view it here");
54 |     JTable specTable;
55 |
56 |     public BankApp() {
57 |         setLayout(new BorderLayout());
58 |         add(buildHeader(), BorderLayout.NORTH);
59 |
60 |         tabs.setFont(new Font("SansSerif", Font.BOLD, 14));
61 |         tabs.addTab("Dashboard", buildDashboard());
62 |         tabs.addTab("Operations", buildOperations());
63 |         tabs.addTab("CSV Import", buildCsvTab());
64 |         tabs.addTab("Specification", buildSpecTab());
65 |         add(tabs, BorderLayout.CENTER);
66 |
67 |         setPreferredSize(new Dimension(1000, 700));
68 |         refreshDashboard();
69 |     }
70 |
71 |     // ------------------------------------------------------------------ header
```

```
 72 |     private JPanel buildHeader() {
 73 |         JPanel header = new JPanel(new BorderLayout(16, 0));
 74 |         header.setBackground(NAVY);
 75 |         header.setBorder(new EmptyBorder(10, 18, 10, 18));
 76 |         try {
 77 |             Image logo = ImageIO.read(new File("logo.png"));
 78 |             int h = 64, w = logo.getWidth(null) * h / logo.getHeight(null);
 79 |             header.add(new JLabel(new ImageIcon(logo.getScaledInstance(w, h, Image.SCALE_SMOOTH))),
 80 |                     BorderLayout.WEST);
 81 |         } catch (Exception ignored) {
 82 |             // the application also works without a logo file
 83 |         }
 84 |         JPanel text = new JPanel(new GridLayout(2, 1));
 85 |         text.setOpaque(false);
 86 |         JLabel title = new JLabel("Bank Management System");
 87 |         title.setFont(new Font("SansSerif", Font.BOLD, 26));
 88 |         title.setForeground(Color.WHITE);
 89 |         JLabel sub = new JLabel(bank.getBankName() + "  |  Second Year BTech Project  |  Sri Sri
    |             University");
 90 |         sub.setFont(new Font("SansSerif", Font.PLAIN, 13));
 91 |         sub.setForeground(new Color(0xBFD3E6));
 92 |         text.add(title);
 93 |         text.add(sub);
 94 |         header.add(text, BorderLayout.CENTER);
 95 |         return header;
 96 |     }
 97 |
 98 |     // ------------------------------------------------------------ dashboard
 99 |     private JPanel buildDashboard() {
100 |         JPanel page = new JPanel(new BorderLayout(0, 14));
101 |         page.setBorder(new EmptyBorder(16, 18, 16, 18));
102 |
103 |         JPanel cards = new JPanel(new GridLayout(1, 4, 14, 0));
104 |         cards.add(card("Total Accounts", lblAccounts, NAVY));
105 |         cards.add(card("Savings Accounts", lblSavings, TEAL));
106 |         cards.add(card("Current Accounts", lblCurrent, new Color(0xE0, 0x9F, 0x1F)));
107 |         cards.add(card("Total Balance", lblTotal, new Color(0x6A, 0x4C, 0x93)));
108 |         page.add(cards, BorderLayout.NORTH);
109 |
110 |         JTable table = styledTable(accountModel);
111 |         JScrollPane scroll = scrollOf(table);
112 |         emptyHint.setFont(new Font("SansSerif", Font.ITALIC, 14));
113 |         emptyHint.setForeground(Color.GRAY);
114 |         JPanel center = new JPanel(new BorderLayout());
115 |         center.add(scroll, BorderLayout.CENTER);
116 |         center.add(emptyHint, BorderLayout.SOUTH);
117 |         center.setBorder(javax.swing.BorderFactory.createTitledBorder("All Accounts"));
118 |         page.add(center, BorderLayout.CENTER);
119 |         return page;
120 |     }
121 |
122 |     private JPanel card(String caption, JLabel value, Color color) {
123 |         JPanel p = new JPanel(new GridLayout(2, 1));
124 |         p.setBackground(Color.WHITE);
125 |         p.setBorder(new javax.swing.border.CompoundBorder(new LineBorder(color, 2, true), new
    |             EmptyBorder(10, 14, 10, 14)));
126 |         JLabel c = new JLabel(caption);
127 |         c.setFont(new Font("SansSerif", Font.BOLD, 13));
128 |         c.setForeground(Color.DARK_GRAY);
129 |         value.setForeground(color);
130 |         p.add(c);
131 |         p.add(value);
132 |         return p;
133 |     }
134 |
135 |     // ------------------------------------------------------------ operations
136 |     private JPanel buildOperations() {
137 |         JPanel page = new JPanel(new BorderLayout(0, 14));
138 |         page.setBorder(new EmptyBorder(16, 18, 16, 18));
139 |
140 |         JPanel form = new JPanel(new GridBagLayout());
141 |         form.setBorder(javax.swing.BorderFactory.createTitledBorder("Account Details"));
142 |         GridBagConstraints g = new GridBagConstraints();
143 |         g.insets = new Insets(8, 10, 8, 10);
144 |         g.anchor = GridBagConstraints.WEST;
```

```
145 |          addRow(form, g, 0, "Account Type", typeBox, "Account Holder Name", holderField);
146 |          addRow(form, g, 1, "Account Number", numberField, "Amount (Rs.)", amountField);
147 |          page.add(form, BorderLayout.NORTH);
148 |
149 |          JPanel buttons = new JPanel(new FlowLayout(FlowLayout.LEFT, 12, 6));
150 |          buttons.add(btnCreate);
151 |          buttons.add(btnDeposit);
152 |          buttons.add(btnWithdraw);
153 |          buttons.add(btnBalance);
154 |
155 |          messageArea.setEditable(false);
156 |          messageArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
157 |          messageArea.setMargin(new Insets(8, 10, 8, 10));
158 |          JPanel log = new JPanel(new BorderLayout());
159 |          log.setBorder(javax.swing.BorderFactory.createTitledBorder("Messages"));
160 |          log.add(new JScrollPane(messageArea), BorderLayout.CENTER);
161 |
162 |          JPanel lower = new JPanel(new BorderLayout(0, 8));
163 |          lower.add(buttons, BorderLayout.NORTH);
164 |          lower.add(log, BorderLayout.CENTER);
165 |          page.add(lower, BorderLayout.CENTER);
166 |
167 |          btnCreate.addActionListener(e -> doCreate());
168 |          btnDeposit.addActionListener(e -> doDeposit());
169 |          btnWithdraw.addActionListener(e -> doWithdraw());
170 |          btnBalance.addActionListener(e -> doBalance());
171 |          return page;
172 |      }
173 |
174 |      private void addRow(JPanel p, GridBagConstraints g, int row, String l1, JComponent c1, String l2,
    |          JComponent c2) {
175 |          g.gridy = row;
176 |          g.gridx = 0; p.add(new JLabel(l1), g);
177 |          g.gridx = 1; p.add(c1, g);
178 |          g.gridx = 2; p.add(new JLabel(l2), g);
179 |          g.gridx = 3; p.add(c2, g);
180 |      }
181 |
182 |      void doCreate() {
183 |          try {
184 |              String holder = holderField.getText().trim();
185 |              if (holder.isEmpty()) {
186 |                  throw new BankException("Please enter the account holder name.");
187 |              }
188 |              Account a = bank.createAccount((String) typeBox.getSelectedItem(), holder, readAmount());
189 |              numberField.setText(a.getAccountNumber());
190 |              say("SUCCESS  " + a.getAccountType() + " account " + a.getAccountNumber()
191 |                      + " created for " + holder + ".");
192 |          } catch (BankException ex) {
193 |              say("REJECTED " + ex.getMessage());
194 |          } catch (NumberFormatException ex) {
195 |              say("REJECTED Please enter a valid amount.");
196 |          }
197 |          refreshDashboard();
198 |      }
199 |
200 |      void doDeposit() {
201 |          try {
202 |              double bal = bank.deposit(numberField.getText().trim(), readAmount());
203 |              say("SUCCESS  Deposit done. New balance: " + money(bal));
204 |          } catch (BankException ex) {
205 |              say("REJECTED " + ex.getMessage());
206 |          } catch (NumberFormatException ex) {
207 |              say("REJECTED Please enter a valid amount.");
208 |          }
209 |          refreshDashboard();
210 |      }
211 |
212 |      void doWithdraw() {
213 |          try {
214 |              double bal = bank.withdraw(numberField.getText().trim(), readAmount());
215 |              say("SUCCESS  Withdrawal done. New balance: " + money(bal));
216 |          } catch (BankException ex) {
217 |              say("REJECTED " + ex.getMessage());
218 |          } catch (NumberFormatException ex) {
```

```
219 |             say("REJECTED Please enter a valid amount.");
220 |         }
221 |         refreshDashboard();
222 |     }
223 |
224 |     void doBalance() {
225 |         try {
226 |             say("BALANCE  " + numberField.getText().trim() + " = "
227 |                     + money(bank.checkBalance(numberField.getText().trim())));
228 |         } catch (BankException ex) {
229 |             say("REJECTED " + ex.getMessage());
230 |         }
231 |     }
232 |
233 |     private double readAmount() {
234 |         return Double.parseDouble(amountField.getText().trim());
235 |     }
236 |
237 |     private void say(String text) {
238 |         messageArea.append(text + "\n");
239 |     }
240 |
241 |     // ------------------------------------------------------------ CSV import
242 |     private JPanel buildCsvTab() {
243 |         JPanel page = new JPanel(new BorderLayout(0, 12));
244 |         page.setBorder(new EmptyBorder(16, 18, 16, 18));
245 |
246 |         JPanel top = new JPanel(new BorderLayout(10, 6));
247 |         JLabel info = new JLabel("<html>Choose a transactions file. Required columns: "
248 |                 + "<b>Action, AccountType, HolderName, AccountNumber, Amount</b> "
249 |                 + "(actions: CREATE, DEPOSIT, WITHDRAW, BALANCE)</html>");
250 |         csvPathField.setEditable(false);
251 |         csvPathField.setText("No file selected");
252 |         top.add(info, BorderLayout.NORTH);
253 |         top.add(csvPathField, BorderLayout.CENTER);
254 |         top.add(btnChooseCsv, BorderLayout.EAST);
255 |         page.add(top, BorderLayout.NORTH);
256 |
257 |         JTable table = styledTable(csvModel);
258 |         table.setRowHeight(23);
259 |         table.getColumnModel().getColumn(0).setPreferredWidth(40);
260 |         table.getColumnModel().getColumn(1).setPreferredWidth(90);
261 |         table.getColumnModel().getColumn(2).setPreferredWidth(90);
262 |         table.getColumnModel().getColumn(3).setPreferredWidth(110);
263 |         table.getColumnModel().getColumn(4).setPreferredWidth(90);
264 |         table.getColumnModel().getColumn(5).setPreferredWidth(420);
265 |         table.getColumnModel().getColumn(4).setCellRenderer(new StatusRenderer());
266 |         page.add(scrollOf(table), BorderLayout.CENTER);
267 |
268 |         csvSummary.setFont(new Font("SansSerif", Font.BOLD, 14));
269 |         csvSummary.setForeground(NAVY);
270 |         page.add(csvSummary, BorderLayout.SOUTH);
271 |
272 |         btnChooseCsv.addActionListener(e -> {
273 |             JFileChooser chooser = new JFileChooser(".");
274 |             if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
275 |                 processCsv(chooser.getSelectedFile());
276 |             }
277 |         });
278 |         return page;
279 |     }
280 |
281 |     void processCsv(File file) {
282 |         csvPathField.setText(file.getAbsolutePath());
283 |         csvModel.setRowCount(0);
284 |         try {
285 |             List<CsvProcessor.Result> results = CsvProcessor.process(file, bank);
286 |             int ok = 0, bad = 0;
287 |             for (CsvProcessor.Result r : results) {
288 |                 csvModel.addRow(new Object[]{r.row, r.action, r.account, r.amount, r.status,
    |                     r.message});
289 |                 if ("Success".equals(r.status)) ok++; else bad++;
290 |             }
291 |             csvSummary.setForeground(NAVY);
292 |             csvSummary.setText("Rows processed: " + results.size() + "   |   Successful: " + ok
```

```
293 |                     + "   |   Rejected: " + bad + "   |   Total balance now: " +
    |                         money(bank.getTotalBalance()));
294 |         } catch (Exception ex) {
295 |             csvSummary.setForeground(RED);
296 |             csvSummary.setText("Could not read the file: " + ex.getMessage());
297 |         }
298 |         refreshDashboard();
299 |     }
300 |
301 |     // ---------------------------------------------------- specification tab
302 |     private JPanel buildSpecTab() {
303 |         JPanel page = new JPanel(new BorderLayout(0, 12));
304 |         page.setBorder(new EmptyBorder(16, 18, 16, 18));
305 |         JPanel top = new JPanel(new BorderLayout(10, 6));
306 |         specPathField.setEditable(false);
307 |         specPathField.setText("No file selected");
308 |         top.add(new JLabel("View the project specification (features, OOP concepts and classes) stored
    |             as a CSV file."),
309 |                 BorderLayout.NORTH);
310 |         top.add(specPathField, BorderLayout.CENTER);
311 |         top.add(btnLoadSpec, BorderLayout.EAST);
312 |         page.add(top, BorderLayout.NORTH);
313 |         specTable = styledTable(specModel);
314 |         page.add(scrollOf(specTable), BorderLayout.CENTER);
315 |
316 |         btnLoadSpec.addActionListener(e -> {
317 |             JFileChooser chooser = new JFileChooser(".");
318 |             if (chooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
319 |                 loadSpec(chooser.getSelectedFile());
320 |             }
321 |         });
322 |         return page;
323 |     }
324 |
325 |     void loadSpec(File file) {
326 |         specPathField.setText(file.getAbsolutePath());
327 |         try {
328 |             List<String[]> rows = CsvProcessor.readAll(file);
329 |             String[] head = rows.get(0);
330 |             List<Integer> show = new ArrayList<Integer>();
331 |             for (int i = 0; i < head.length; i++) {
332 |                 if (!head[i].equals("Attributes") && !head[i].equals("Methods")) show.add(i);
333 |             }
334 |             Object[] columns = new Object[show.size()];
335 |             for (int i = 0; i < columns.length; i++) columns[i] = head[show.get(i)];
336 |             specModel.setDataVector(new Object[0][0], columns);
337 |             for (int r = 1; r < rows.size(); r++) {
338 |                 Object[] line = new Object[columns.length];
339 |                 for (int i = 0; i < columns.length; i++) line[i] = rows.get(r)[show.get(i)];
340 |                 specModel.addRow(line);
341 |             }
342 |             String[] names = {"Category", "ID", "Name", "Description", "OOP Concept Applied", "Related
    |                 Classes"};
343 |             int[] widths = {95, 45, 125, 360, 150, 205};
344 |             for (int i = 0; i < columns.length; i++) {
345 |                 for (int k = 0; k < names.length; k++) {
346 |                     if (names[k].equals(columns[i]))
    |                         specTable.getColumnModel().getColumn(i).setPreferredWidth(widths[k]);
347 |                 }
348 |             }
349 |         } catch (Exception ex) {
350 |             specPathField.setText("Could not read the file: " + ex.getMessage());
351 |         }
352 |     }
353 |
354 |     // ------------------------------------------------------------- helpers
355 |     void refreshDashboard() {
356 |         accountModel.setRowCount(0);
357 |         int savings = 0, current = 0;
358 |         for (Account a : bank.getAllAccounts()) {
359 |             accountModel.addRow(new Object[]{a.getAccountNumber(), a.getHolderName(),
    |                 a.getAccountType(),
360 |                     String.format("%,.2f", a.getBalance())});
361 |             if (a instanceof SavingsAccount) savings++; else current++;
362 |         }
```

```
363 |          lblAccounts.setText(String.valueOf(savings + current));
364 |          lblSavings.setText(String.valueOf(savings));
365 |          lblCurrent.setText(String.valueOf(current));
366 |          lblTotal.setText(money(bank.getTotalBalance()));
367 |          lblTotal.setFont(new Font("SansSerif", Font.BOLD, 20));
368 |          emptyHint.setVisible(accountModel.getRowCount() == 0);
369 |      }
370 |
371 |      static JScrollPane scrollOf(JTable t) {
372 |          JScrollPane sp = new JScrollPane(t);
373 |          sp.setColumnHeaderView(t.getTableHeader());
374 |          return sp;
375 |      }
376 |
377 |      static String money(double v) {
378 |          return String.format("Rs. %,.2f", v);
379 |      }
380 |
381 |      static JLabel valueLabel() {
382 |          JLabel l = new JLabel("0");
383 |          l.setFont(new Font("SansSerif", Font.BOLD, 26));
384 |          return l;
385 |      }
386 |
387 |      static JButton button(String text, Color color) {
388 |          JButton b = new JButton(text);
389 |          b.setBackground(color);
390 |          b.setForeground(Color.WHITE);
391 |          b.setFont(new Font("SansSerif", Font.BOLD, 13));
392 |          b.setFocusPainted(false);
393 |          b.setBorder(new EmptyBorder(9, 18, 9, 18));
394 |          b.setOpaque(true);
395 |          return b;
396 |      }
397 |
398 |      static DefaultTableModel model(String... columns) {
399 |          return new DefaultTableModel(columns, 0) {
400 |              @Override
401 |              public boolean isCellEditable(int row, int column) {
402 |                  return false;
403 |              }
404 |          };
405 |      }
406 |
407 |      static JTable styledTable(DefaultTableModel m) {
408 |          JTable t = new JTable(m);
409 |          t.setRowHeight(28);
410 |          t.setFont(new Font("SansSerif", Font.PLAIN, 13));
411 |          t.setShowGrid(true);
412 |          t.setGridColor(new Color(0xD5DDE8));
413 |          t.setFillsViewportHeight(true);
414 |          JTableHeader h = t.getTableHeader();
415 |          h.setReorderingAllowed(false);
416 |          h.setDefaultRenderer(new DefaultTableCellRenderer() {
417 |              @Override
418 |              public Component getTableCellRendererComponent(JTable tb, Object v, boolean s, boolean f,
    |                  int r, int c) {
419 |                  JLabel l = (JLabel) super.getTableCellRendererComponent(tb, v, s, f, r, c);
420 |                  l.setBackground(NAVY);
421 |                  l.setForeground(Color.WHITE);
422 |                  l.setFont(new Font("SansSerif", Font.BOLD, 13));
423 |                  l.setHorizontalAlignment(SwingConstants.CENTER);
424 |                  l.setBorder(new EmptyBorder(6, 6, 6, 6));
425 |                  return l;
426 |              }
427 |          });
428 |          t.setDefaultRenderer(Object.class, new DefaultTableCellRenderer() {
429 |              @Override
430 |              public Component getTableCellRendererComponent(JTable tb, Object v, boolean s, boolean f,
    |                  int r, int c) {
431 |                  JLabel l = (JLabel) super.getTableCellRendererComponent(tb, v, s, f, r, c);
432 |                  l.setBackground(r % 2 == 0 ? Color.WHITE : LIGHT);
433 |                  l.setForeground(Color.BLACK);
434 |                  l.setBorder(new EmptyBorder(0, 8, 0, 8));
435 |                  return l;
```

```
            }
        });
        return t;
    }

    /** Colours the Status column green / red. */
    static class StatusRenderer extends DefaultTableCellRenderer {
        @Override
        public Component getTableCellRendererComponent(JTable tb, Object v, boolean s, boolean f, int r,
             int c) {
            JLabel l = (JLabel) super.getTableCellRendererComponent(tb, v, s, f, r, c);
            l.setBackground(r % 2 == 0 ? Color.WHITE : LIGHT);
            l.setForeground("Success".equals(v) ? GREEN : RED);
            l.setFont(new Font("SansSerif", Font.BOLD, 13));
            l.setHorizontalAlignment(SwingConstants.CENTER);
            return l;
        }
    }

    public static void main(String[] args) {
        try {
            for (UIManager.LookAndFeelInfo info : UIManager.getInstalledLookAndFeels()) {
                if ("Nimbus".equals(info.getName())) {
                    UIManager.setLookAndFeel(info.getClassName());
                    break;
                }
            }
        } catch (Exception ignored) {
            // fall back to the default look and feel
        }
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Bank Management System - Sri Sri University");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setContentPane(new BankApp());
            frame.pack();
            frame.setLocationRelativeTo(null);
            frame.setVisible(true);
        });
    }
}
```

File: TestRunner.java

```
import java.io.PrintWriter;
import java.lang.reflect.Modifier;
import java.util.ArrayList;
import java.util.List;

/** Runs the designed test cases and writes the results to a CSV file. */
public class TestRunner {

    interface Check { String run() throws Exception; }

    static final List<String[]> rows = new ArrayList<String[]>();

    static void test(String id, String description, String expected, Check check) {
        String actual;
        try {
            actual = check.run();
        } catch (Exception e) {
            actual = e.getClass().getSimpleName();
        }
        rows.add(new String[]{id, description, expected, actual, expected.equals(actual) ? "Pass" :
            "Fail"});
    }

    public static void main(String[] args) throws Exception {
        final Bank bank = new Bank("Test Bank");
        final Account s = bank.createAccount("savings", "Test Savings", 5000);
        final Account c = bank.createAccount("current", "Test Current", 2000);

        test("T01", "Create savings account with Rs. 5,000", "5000.0",
                () -> String.valueOf(bank.checkBalance(s.getAccountNumber())));
        test("T02", "Deposit Rs. 1,500 into savings account", "6500.0",
```

```
                () -> String.valueOf(bank.deposit(s.getAccountNumber(), 1500)));
        test("T03", "Withdraw Rs. 2,000 from savings account", "4500.0",
                () -> String.valueOf(bank.withdraw(s.getAccountNumber(), 2000)));
        test("T04", "Withdraw below minimum balance (savings)", "InsufficientFundsException",
                () -> String.valueOf(bank.withdraw(s.getAccountNumber(), 4300)));
        test("T05", "Withdraw Rs. 5,000 from current (uses overdraft)", "-3000.0",
                () -> String.valueOf(bank.withdraw(c.getAccountNumber(), 5000)));
        test("T06", "Withdraw beyond overdraft limit (current)", "InsufficientFundsException",
                () -> String.valueOf(bank.withdraw(c.getAccountNumber(), 8000)));
        test("T07", "Deposit a negative amount", "InvalidAmountException",
                () -> String.valueOf(bank.deposit(s.getAccountNumber(), -100)));
        test("T08", "Access a non-existent account", "AccountNotFoundException",
                () -> String.valueOf(bank.checkBalance("ACC9999")));
        test("T09", "Create account with unknown type", "BankException",
                () -> String.valueOf(bank.createAccount("fixed", "X", 100)));
        test("T10", "Polymorphism: same withdraw call, different rule", "Rejected/Success", () -> {
            Account a = new SavingsAccount("T1", "A", 2000);
            Account b = new CurrentAccount("T2", "B", 2000);
            String first, second;
            try { a.withdraw(1800); first = "Success"; } catch (BankException e) { first = "Rejected"; }
            try { b.withdraw(1800); second = "Success"; } catch (BankException e) { second = "Rejected";
                 }
            return first + "/" + second;
        });
        test("T11", "Abstraction: Account class is abstract", "true",
                () -> String.valueOf(Modifier.isAbstract(Account.class.getModifiers())));
        test("T12", "Encapsulation: balance field is private", "true",
                () ->
                    String.valueOf(Modifier.isPrivate(Account.class.getDeclaredField("balance").getModif
                    iers())));

        PrintWriter out = new PrintWriter(args[0]);
        out.println("id,description,expected,actual,result");
        for (String[] r : rows) {
            out.println(r[0] + ",\"" + r[1] + "\"," + r[2] + "," + r[3] + "," + r[4]);
        }
        out.close();
        int pass = 0;
        for (String[] r : rows) { System.out.println(String.join(" | ", r)); if (r[4].equals("Pass"))
            pass++; }
        System.out.println(pass + " / " + rows.size() + " tests passed");
    }
}
```

File: Simulation.java

```
import java.io.PrintWriter;
import java.util.List;
import java.util.Random;

/** Applies 60 random transactions to 8 accounts and records every result. */
public class Simulation {

    public static void main(String[] args) throws Exception {
        String dir = args[0];
        Bank bank = new Bank("BTech National Bank");
        String[][] people = {
                {"Aarav", "savings", "25000"}, {"Diya", "savings", "18000"}, {"Rohan", "current",
                    "40000"},
                {"Sneha", "savings", "9000"}, {"Kabir", "current", "60000"}, {"Meera", "savings",
                    "32000"},
                {"Vikram", "current", "15000"}, {"Ananya", "savings", "12000"}};
        double[] amounts = {500, 1000, 2000, 3500, 5000, 8000, 12000, 20000, 35000};

        PrintWriter ledger = new PrintWriter(dir + "/ledger.csv");
        PrintWriter running = new PrintWriter(dir + "/running.csv");
        ledger.println("acc,type,kind,amount,status");
        running.println("total");

        for (String[] p : people) {
            Account a = bank.createAccount(p[1], p[0], Double.parseDouble(p[2]));
            ledger.println(a.getAccountNumber() + "," + a.getAccountType() + ",Opening," + p[2] +
                ",Success");
        }
```

```
        running.println(bank.getTotalBalance());

        Random random = new Random(7);
        List<Account> accounts = bank.getAllAccounts();
        for (int i = 0; i < 60; i++) {
            Account acc = accounts.get(random.nextInt(accounts.size()));
            boolean deposit = random.nextInt(3) == 0;           // about one third deposits
            double amount = amounts[random.nextInt(amounts.length)];
            String status = "Success";
            try {
                if (deposit) acc.deposit(amount); else acc.withdraw(amount);
            } catch (BankException e) {
                status = "Rejected";
            }
            ledger.println(acc.getAccountNumber() + "," + acc.getAccountType() + ","
                    + (deposit ? "Deposit" : "Withdrawal") + "," + (int) amount + "," + status);
            running.println(bank.getTotalBalance());
        }
        ledger.close();
        running.close();

        PrintWriter out = new PrintWriter(dir + "/accounts.csv");
        out.println("number,holder,type,balance");
        for (Account a : accounts) {
            out.println(a.getAccountNumber() + "," + a.getHolderName() + "," + a.getAccountType() + ","
                + a.getBalance());
        }
        out.close();
        System.out.println("Simulation finished. Closing total = " + bank.getTotalBalance());
    }
}
```

Part 2: Python Source Code

File: bank_management_system.py

```
"""
Bank Management System - Python version
Sri Sri University | Second Year BTech Project

Demonstrates the four pillars of OOP:
    Encapsulation, Inheritance, Polymorphism and Abstraction.

Classes : Account (abstract), SavingsAccount, CurrentAccount, Bank
Features: create accounts, deposit, withdraw, check balance, import a CSV file

How to run
    python bank_management_system.py                      -> interactive menu
    python bank_management_system.py data.csv             -> process a transactions CSV file
    python bank_management_system.py data.csv -o out.csv  -> ... and save the results

CSV columns: Action, AccountType, HolderName, AccountNumber, Amount
Actions    : CREATE, DEPOSIT, WITHDRAW, BALANCE
"""

import csv
import sys
from abc import ABC, abstractmethod


# ---------------------------------------------------------------------------
# Exceptions
# ---------------------------------------------------------------------------
class BankError(Exception):
    """Base class for every banking related error."""


class InvalidAmountError(BankError):
    """Raised when an amount is zero, negative or otherwise not acceptable."""


class InsufficientFundsError(BankError):
    """Raised when a withdrawal breaks the rules of the account type."""


class AccountNotFoundError(BankError):
    """Raised when an account number does not exist in the bank."""


# ---------------------------------------------------------------------------
# Object model
# ---------------------------------------------------------------------------
class Account(ABC):
    """Abstract base class: common data and interface of every account."""

    def __init__(self, account_number, holder_name, opening_balance=0.0):
        if opening_balance < 0:
            raise InvalidAmountError("Opening balance cannot be negative.")
        self.__account_number = account_number      # private (Encapsulation)
        self.__holder_name = holder_name            # private
        self.__balance = float(opening_balance)     # private
        self._transactions = []                     # protected by convention
        self._log("Opening", opening_balance, "Success")

    # ---- read-only access through properties (no setter for balance) ----
    @property
    def account_number(self):
        return self.__account_number

    @property
    def holder_name(self):
        return self.__holder_name

```

```
    @property
    def balance(self):
        return self.__balance

    @property
    def transactions(self):
        return list(self._transactions)            # a copy keeps the original safe

    # ---- behaviour shared by all accounts ----
    def deposit(self, amount):
        self._validate(amount)
        self.__balance += amount
        self._log("Deposit", amount, "Success")
        return self.__balance

    # ---- protected helpers for subclasses ----
    def _debit(self, amount):
        self.__balance -= amount
        self._log("Withdrawal", amount, "Success")
        return self.__balance

    def _reject(self, amount):
        self._log("Withdrawal", amount, "Rejected")

    @staticmethod
    def _validate(amount):
        if amount <= 0:
            raise InvalidAmountError("Amount must be greater than zero.")

    def _log(self, kind, amount, status):
        self._transactions.append({"type": kind, "amount": float(amount),
                                   "status": status, "balance_after": self.__balance})

    # ---- Abstraction: every subclass MUST implement these ----
    @abstractmethod
    def withdraw(self, amount):
        """Each account type applies its own withdrawal rule."""

    @abstractmethod
    def account_type(self):
        """Return the name of the account type."""

    def __str__(self):
        return (f"{self.account_type():<8} | {self.account_number} | "
                f"{self.holder_name:<22} | Rs. {self.balance:>12,.2f}")


class SavingsAccount(Account):
    """Earns interest and must always keep a minimum balance."""

    def __init__(self, account_number, holder_name, opening_balance=0.0):
        super().__init__(account_number, holder_name, opening_balance)
        self.interest_rate = 0.04
        self.minimum_balance = 500.0

    def withdraw(self, amount):                    # Polymorphism: savings rule
        self._validate(amount)
        if self.balance - amount < self.minimum_balance:
            self._reject(amount)
            raise InsufficientFundsError(
                f"Minimum balance of Rs. {self.minimum_balance:,.0f} must be maintained.")
        return self._debit(amount)

    def add_interest(self):
        """Credits one month of interest and returns the amount credited."""
        interest = self.balance * self.interest_rate / 12
        self.deposit(interest)
        return interest

    def account_type(self):
        return "Savings"


class CurrentAccount(Account):
    """Business account that allows withdrawals up to an overdraft limit."""
```

```
143 |
144 |     def __init__(self, account_number, holder_name, opening_balance=0.0):
145 |         super().__init__(account_number, holder_name, opening_balance)
146 |         self.overdraft_limit = 10000.0
147 |
148 |     def withdraw(self, amount):                    # Polymorphism: current-account rule
149 |         self._validate(amount)
150 |         if self.balance - amount < -self.overdraft_limit:
151 |             self._reject(amount)
152 |             raise InsufficientFundsError(
153 |                 f"Overdraft limit of Rs. {self.overdraft_limit:,.0f} exceeded.")
154 |         return self._debit(amount)
155 |
156 |     def account_type(self):
157 |         return "Current"
158 |
159 |
160 | class Bank:
161 |     """The bank: owns every account and coordinates all operations."""
162 |
163 |     def __init__(self, name):
164 |         self.name = name
165 |         self.__accounts = {}                       # private storage
166 |         self.__next_number = 1001
167 |
168 |     def create_account(self, kind, holder_name, opening_balance=0.0):
169 |         number = f"ACC{self.__next_number}"
170 |         kind = kind.strip().lower()
171 |         if kind == "savings":
172 |             account = SavingsAccount(number, holder_name, opening_balance)
173 |         elif kind == "current":
174 |             account = CurrentAccount(number, holder_name, opening_balance)
175 |         else:
176 |             raise BankError("Account type must be SAVINGS or CURRENT.")
177 |         self.__accounts[number] = account
178 |         self.__next_number += 1
179 |         return account
180 |
181 |     def find_account(self, account_number):
182 |         try:
183 |             return self.__accounts[account_number]
184 |         except KeyError:
185 |             raise AccountNotFoundError(f"Account {account_number} not found.")
186 |
187 |     def deposit(self, account_number, amount):
188 |         return self.find_account(account_number).deposit(amount)
189 |
190 |     def withdraw(self, account_number, amount):
191 |         return self.find_account(account_number).withdraw(amount)   # polymorphic call
192 |
193 |     def check_balance(self, account_number):
194 |         return self.find_account(account_number).balance
195 |
196 |     def all_accounts(self):
197 |         return list(self.__accounts.values())
198 |
199 |     def total_balance(self):
200 |         return sum(a.balance for a in self.__accounts.values())
201 |
202 |
203 | # ---------------------------------------------------------------------------
204 | # CSV import
205 | # ---------------------------------------------------------------------------
206 | REQUIRED_COLUMNS = ["Action", "AccountType", "HolderName", "AccountNumber", "Amount"]
207 |
208 |
209 | def process_csv(path, bank):
210 |     """Runs every row of a transactions CSV file against the bank.
211 |
212 |     Returns a list of dictionaries: row, action, account, amount, status, message.
213 |     """
214 |     with open(path, newline="", encoding="utf-8-sig") as handle:
215 |         rows = [r for r in csv.reader(handle) if any(f.strip() for f in r)]
216 |     if not rows:
217 |         raise BankError("The file is empty.")
```

```
    header = [h.strip().lower() for h in rows[0]]
    index = {name: (header.index(name.lower()) if name.lower() in header else -1)
             for name in REQUIRED_COLUMNS}
    if index["Action"] < 0 or index["Amount"] < 0:
        raise BankError("This does not look like a transactions file. The first row must "
                        "contain the columns: " + ", ".join(REQUIRED_COLUMNS) + ".")

    def field(row, name):
        i = index[name]
        return row[i].strip() if 0 <= i < len(row) else ""

    results = []
    for number_of_row, row in enumerate(rows[1:], start=1):
        action = field(row, "Action").upper()
        kind, holder = field(row, "AccountType"), field(row, "HolderName")
        number, amount_text = field(row, "AccountNumber"), field(row, "Amount")

        def add(account, amount, status, message):
            results.append({"row": number_of_row, "action": action, "account": account,
                            "amount": amount, "status": status, "message": message})
        try:
            if action in ("CREATE", "DEPOSIT", "WITHDRAW"):
                try:
                    amount = float(amount_text)
                except ValueError:
                    raise BankError("Amount is not a number")
            if action == "CREATE":
                account = bank.create_account(kind, holder, amount)
                add(account.account_number, amount_text, "Success",
                    f"Created {account.account_type()} account for {holder}")
            elif action == "DEPOSIT":
                add(number, amount_text, "Success", f"New balance Rs. {bank.deposit(number,
                    amount):,.2f}")
            elif action == "WITHDRAW":
                add(number, amount_text, "Success", f"New balance Rs. {bank.withdraw(number,
                    amount):,.2f}")
            elif action == "BALANCE":
                add(number, "-", "Success", f"Balance is Rs. {bank.check_balance(number):,.2f}")
            else:
                add(number, amount_text, "Rejected", "Unknown action")
        except BankError as error:
            add(number, amount_text, "Rejected", str(error))
    return results


def print_results(results, bank):
    print(f"{'#':>3}  {'Action':<9} {'Account':<9} {'Amount':>9}  {'Status':<9} Details")
    print("-" * 86)
    for r in results:
        print(f"{r['row']:>3}  {r['action']:<9} {r['account']:<9} {r['amount']:>9}  {r['status']:<9}
            {r['message']}")
    ok = sum(r["status"] == "Success" for r in results)
    print("-" * 86)
    print(f"Rows processed: {len(results)} | Successful: {ok} | Rejected: {len(results) - ok} | "
          f"Total balance now: Rs. {bank.total_balance():,.2f}")


def save_results(results, path):
    with open(path, "w", newline="", encoding="utf-8") as handle:
        writer = csv.writer(handle)
        writer.writerow(["Row", "Action", "Account", "Amount", "Status", "Details"])
        for r in results:
            writer.writerow([r["row"], r["action"], r["account"], r["amount"], r["status"],
                r["message"]])


# ---------------------------------------------------------------------------
# Console menu
# ---------------------------------------------------------------------------
def menu(bank):
    text = ("\n=== BTech National Bank ===\n1. Create account\n2. Deposit\n3. Withdraw\n"
            "4. Check balance\n5. List accounts\n6. Import CSV file\n7. Exit\n")
    while True:
        print(text)
        choice = input("Choose an option (1-7): ").strip()
```

```
289 |         try:
290 |             if choice == "1":
291 |                 kind = input("Type (savings/current): ")
292 |                 name = input("Account holder name: ")
293 |                 account = bank.create_account(kind, name, float(input("Opening balance (Rs.): ")))
294 |                 print(f"Account created -> {account.account_number}")
295 |             elif choice == "2":
296 |                 number = input("Account number: ")
297 |                 print(f"Deposit successful. New balance: Rs. {bank.deposit(number, float(input('Amount
    |                     (Rs.): '))):,.2f}")
298 |             elif choice == "3":
299 |                 number = input("Account number: ")
300 |                 print(f"Withdrawal successful. New balance: Rs. {bank.withdraw(number,
    |                     float(input('Amount (Rs.): '))):,.2f}")
301 |             elif choice == "4":
302 |                 print(f"Balance: Rs. {bank.check_balance(input('Account number: ')):,.2f}")
303 |             elif choice == "5":
304 |                 for account in bank.all_accounts():
305 |                     print(account)
306 |             elif choice == "6":
307 |                 print_results(process_csv(input("Path of the CSV file: ").strip(), bank), bank)
308 |             elif choice == "7":
309 |                 print("Thank you for banking with us.")
310 |                 break
311 |             else:
312 |                 print("Invalid option. Please choose between 1 and 7.")
313 |         except ValueError:
314 |             print("Error: please enter a valid number.")
315 |         except (BankError, OSError) as error:
316 |             print(f"Error: {error}")
317 |
318 |
319 | def main(argv):
320 |     bank = Bank("BTech National Bank")
321 |     if len(argv) >= 2:
322 |         try:
323 |             results = process_csv(argv[1], bank)
324 |         except (BankError, OSError) as error:
325 |             print(f"Error: {error}")
326 |             return 1
327 |         print_results(results, bank)
328 |         if "-o" in argv and argv.index("-o") + 1 < len(argv):
329 |             save_results(results, argv[argv.index("-o") + 1])
330 |             print(f"Results saved to {argv[argv.index('-o') + 1]}")
331 |         return 0
332 |     menu(bank)
333 |     return 0
334 |
335 |
336 | if __name__ == "__main__":
337 |     sys.exit(main(sys.argv))
```